

ACCELERATING THE 2D FAST FOURIER TRANSFORM USING OPENMP AND CUDA

Final project of the course Modern Computing for Physics

Student: Laura Ravagnani

Student ID: 2104271



OVERVIEW



FFT Algorithm
&
Serial
Implementation



OpenMP
Implementation



CUDA
Implementation



Benchmarks

FAST FOURIER TRANSFORM

Fast Fourier Transform (FFT)

Algorithm that computes the Discrete Fourier Transform (DFT) of a sequence

Converts a signal from its original domain (time or space) to a representation in the **frequency domain** and vice versa

Maps a sequence of complex-valued samples x_n , $n = 0, \dots, N - 1$, sampled in the time or space domain, into its frequency domain representation X_k

$$X_k = \sum_{n=0}^{N-1} x_n e^{-i \frac{2\pi}{N} kn}, \quad k = 0, 1, \dots, N - 1$$

FAST FOURIER TRANSFORM

$$X_k = \sum_{n=0}^{N-1} x_n e^{-i \frac{2\pi}{N} kn}, \quad k = 0, 1, \dots, N-1$$

Naive DFT $\longrightarrow O(N^2)$ operations

FFT algorithm (*Divide and Conquer*) $\longrightarrow O(N \log N)$ operations

Radix-2 Cooley-Tukey
algorithm

Divides a DFT of size N into 2 DFTs of size $N/2$ with each **recursive stage** (assuming N is power of 2)
Rearranges the DFT into **2 parts**: sum over the even indices $n=2m$, sum over the odd indices $n=2m+1$

FAST FOURIER TRANSFORM

$$X_k = \sum_{m=0}^{N/2-1} x_{2m} e^{-\frac{2\pi i}{N}(2m)k} + \sum_{m=0}^{N/2-1} x_{2m+1} e^{-\frac{2\pi i}{N}(2m+1)k}$$

$$X_k = \underbrace{\sum_{m=0}^{N/2-1} x_{2m} e^{-\frac{2\pi i}{N/2}mk}}_{\text{DFT of even-indexed part of } x_n} + e^{-\frac{2\pi i}{N}k} \underbrace{\sum_{m=0}^{N/2-1} x_{2m+1} e^{-\frac{2\pi i}{N/2}mk}}_{\text{DFT of odd-indexed part of } x_n} = E_k + e^{-\frac{2\pi i}{N}k} O_k \quad \text{for } k = 0, \dots, \frac{N}{2} - 1$$

twiddle factor

$$X_{k+\frac{N}{2}} = \dots = E_k - e^{-\frac{2\pi i}{N}k} O_k$$

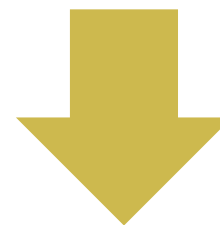
E_k and O_k are calculated in this way for **$k = 0, \dots, N/2 - 1$ only**, thanks to the periodicity of the complex exponential. $X_{k+N/2}$ is also obtained from E_k and O_k

FAST FOURIER TRANSFORM

2-Dimensional FFT

$$X_{k,l} = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} x_{m,n} e^{-i\frac{2\pi}{M}km} e^{-i\frac{2\pi}{N}ln}, \quad k = 0, \dots, M-1, \quad l = 0, \dots, N-1$$

$2N$ FFTs of size N (rows + columns)

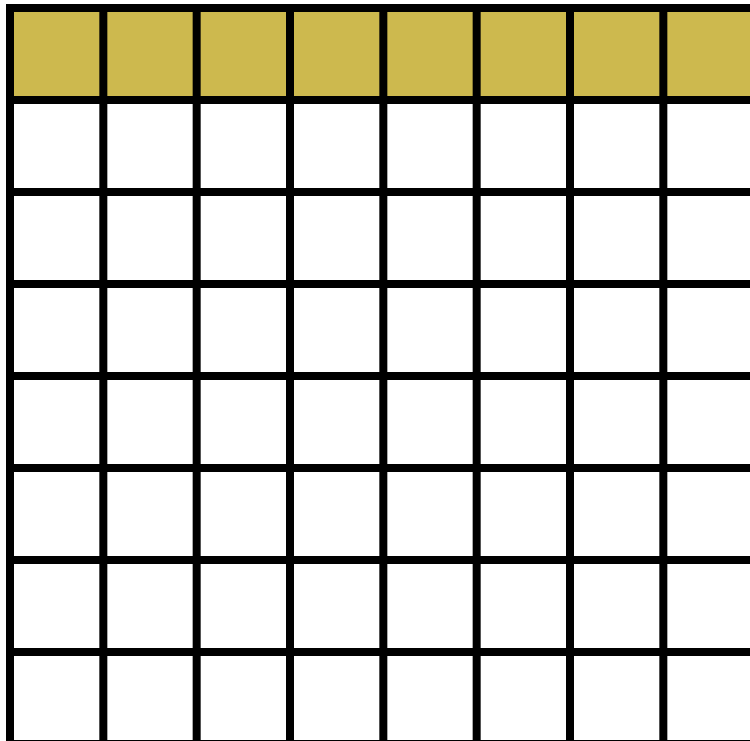


$O(N^2 \log N)$

FAST FOURIER TRANSFORM

2-Dimensional FFT

$$X_{k,l} = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} x_{m,n} e^{-i\frac{2\pi}{M}km} e^{-i\frac{2\pi}{N}ln}, \quad k = 0, \dots, M-1, \quad l = 0, \dots, N-1$$

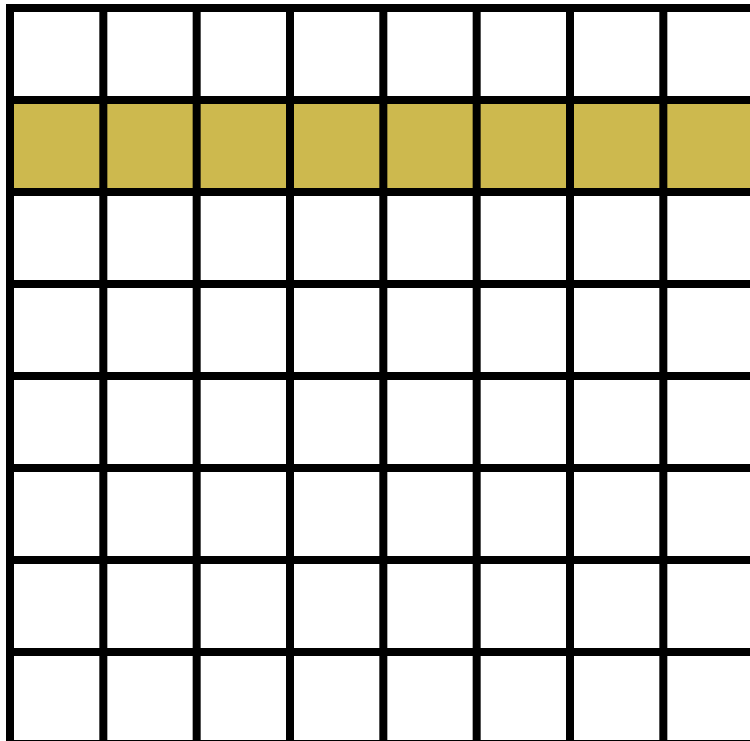


1. N 1D FFT along rows

FAST FOURIER TRANSFORM

2-Dimensional FFT

$$X_{k,l} = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} x_{m,n} e^{-i\frac{2\pi}{M}km} e^{-i\frac{2\pi}{N}ln}, \quad k = 0, \dots, M-1, \quad l = 0, \dots, N-1$$

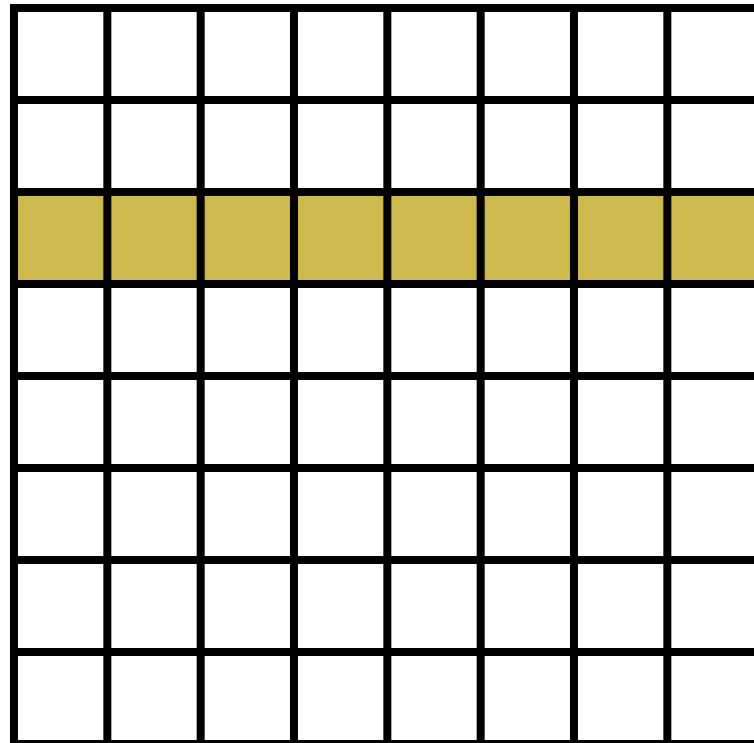


1. N 1D FFT along rows

FAST FOURIER TRANSFORM

2-Dimensional FFT

$$X_{k,l} = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} x_{m,n} e^{-i\frac{2\pi}{M}km} e^{-i\frac{2\pi}{N}ln}, \quad k = 0, \dots, M-1, \quad l = 0, \dots, N-1$$

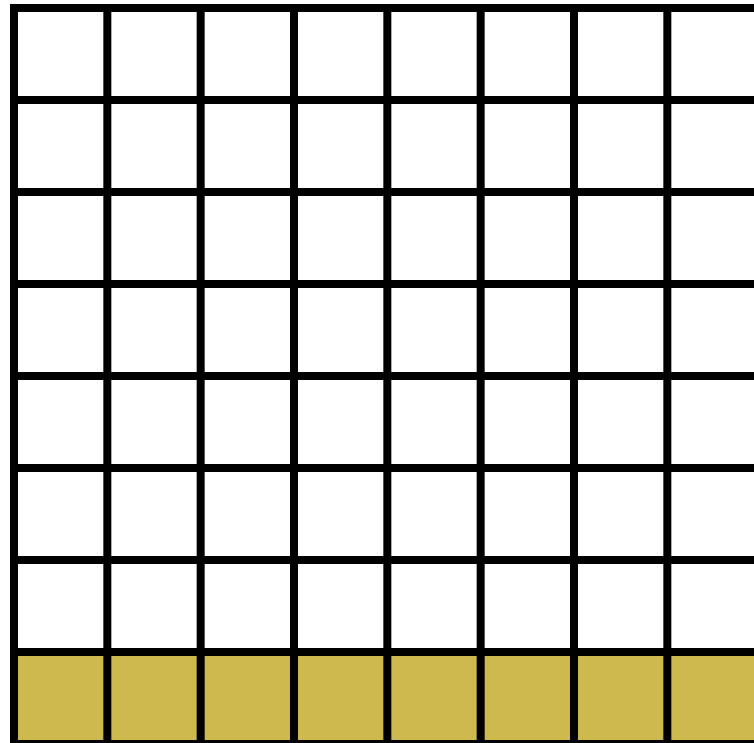


1. N 1D FFT along rows

FAST FOURIER TRANSFORM

2-Dimensional FFT

$$X_{k,l} = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} x_{m,n} e^{-i\frac{2\pi}{M}km} e^{-i\frac{2\pi}{N}ln}, \quad k = 0, \dots, M-1, \quad l = 0, \dots, N-1$$

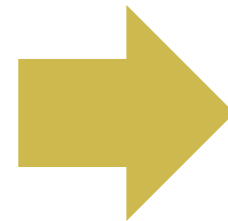
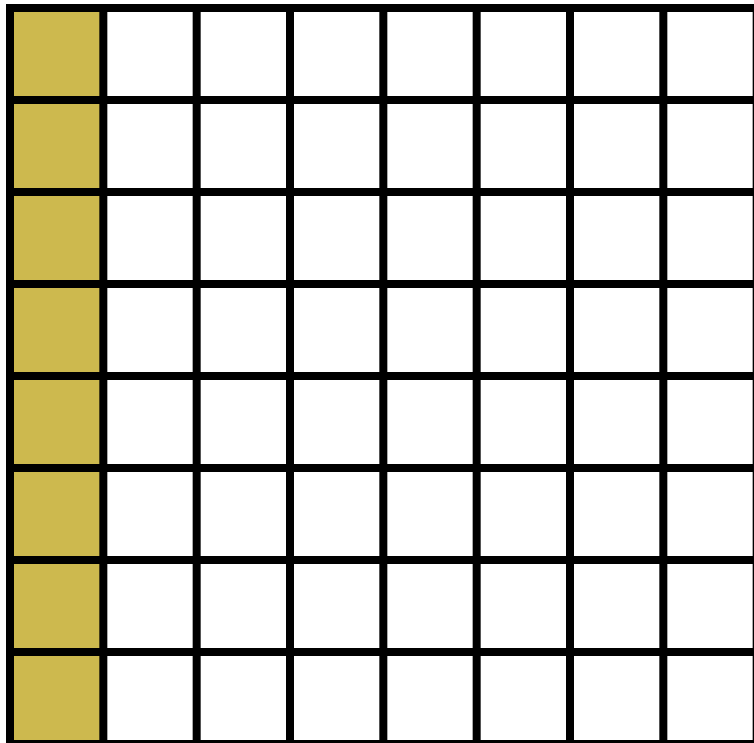


1. N 1D FFT along rows

FAST FOURIER TRANSFORM

2-Dimensional FFT

$$X_{k,l} = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} x_{m,n} e^{-i\frac{2\pi}{M}km} e^{-i\frac{2\pi}{N}ln}, \quad k = 0, \dots, M-1, \quad l = 0, \dots, N-1$$

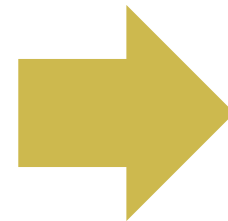
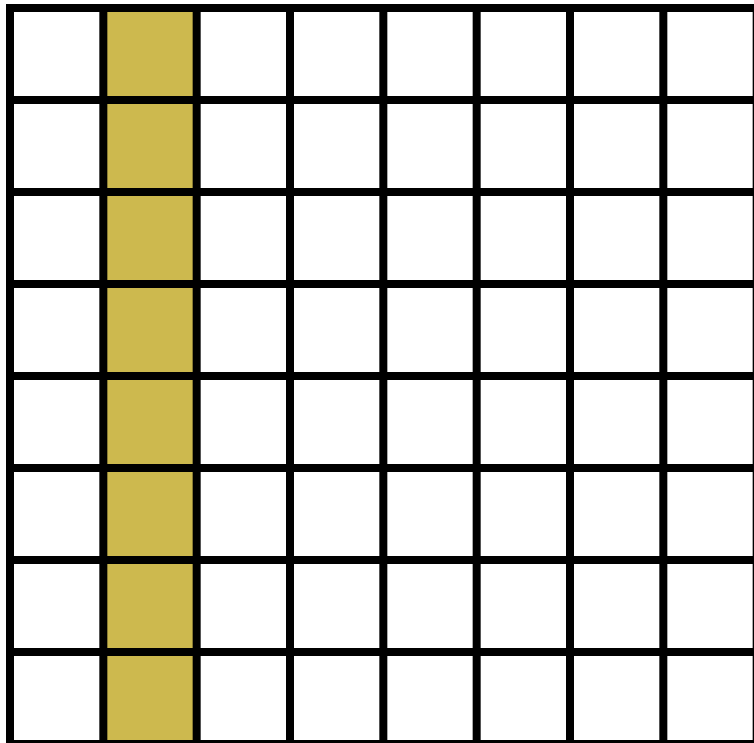


1. N 1D FFT along rows
2. Extract columns + FFT along temporary array

FAST FOURIER TRANSFORM

2-Dimensional FFT

$$X_{k,l} = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} x_{m,n} e^{-i\frac{2\pi}{M}km} e^{-i\frac{2\pi}{N}ln}, \quad k = 0, \dots, M-1, \quad l = 0, \dots, N-1$$

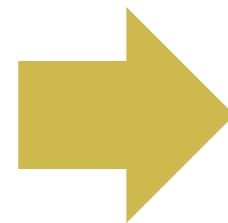
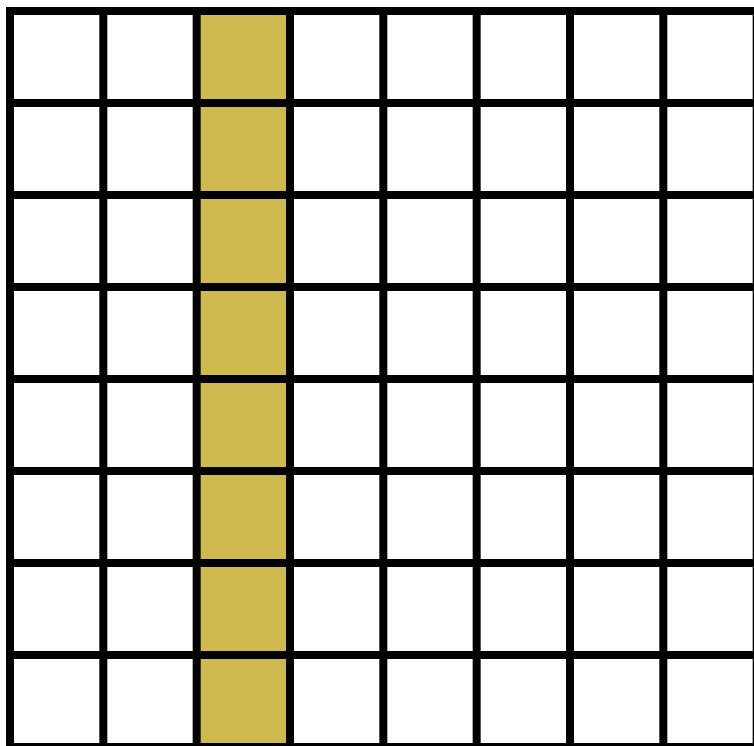


1. N 1D FFT along rows
2. Extract columns + FFT along temporary array

FAST FOURIER TRANSFORM

2-Dimensional FFT

$$X_{k,l} = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} x_{m,n} e^{-i\frac{2\pi}{M}km} e^{-i\frac{2\pi}{N}ln}, \quad k = 0, \dots, M-1, \quad l = 0, \dots, N-1$$

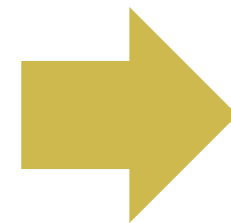
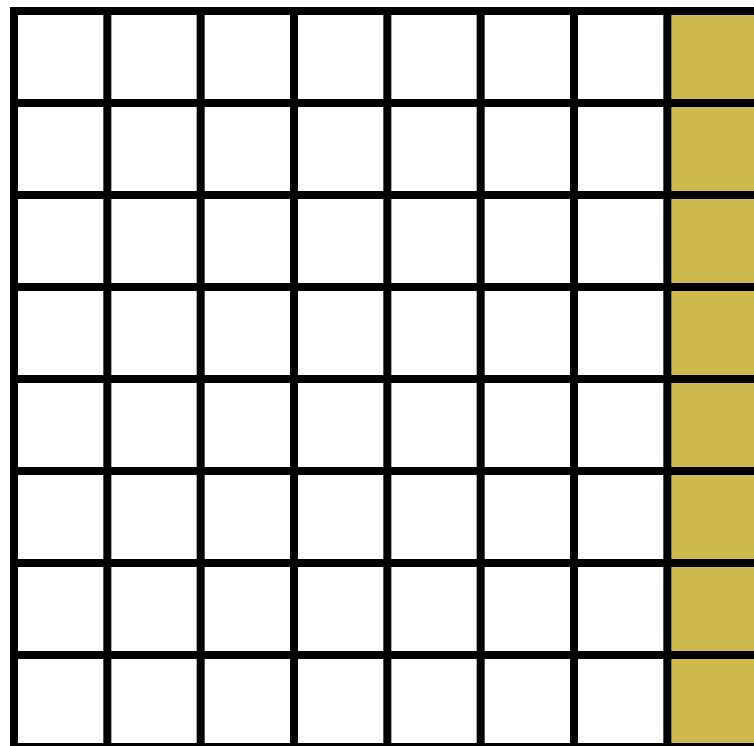


1. N 1D FFT along rows
2. Extract columns + FFT along temporary array

FAST FOURIER TRANSFORM

2-Dimensional FFT

$$X_{k,l} = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} x_{m,n} e^{-i\frac{2\pi}{M}km} e^{-i\frac{2\pi}{N}ln}, \quad k = 0, \dots, M-1, \quad l = 0, \dots, N-1$$

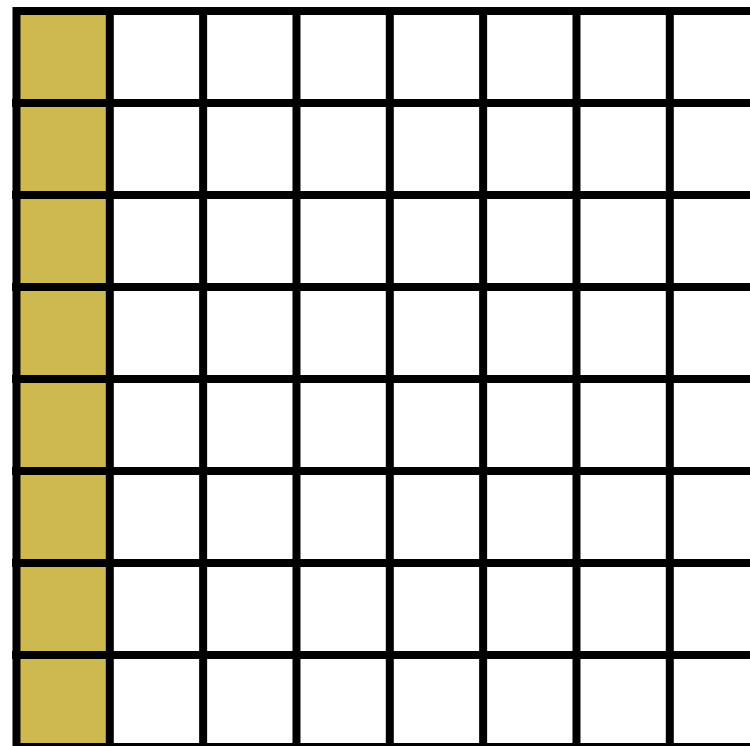
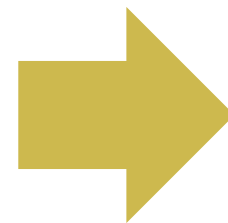
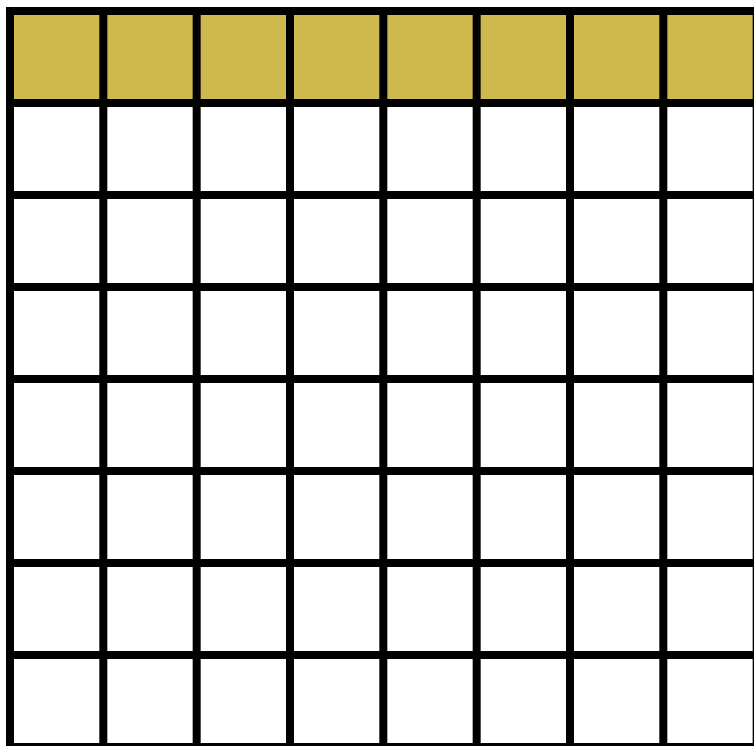


1. N 1D FFT along rows
2. Extract columns + FFT along temporary array

FAST FOURIER TRANSFORM

2-Dimensional FFT

$$X_{k,l} = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} x_{m,n} e^{-i\frac{2\pi}{M}km} e^{-i\frac{2\pi}{N}ln}, \quad k = 0, \dots, M-1, \quad l = 0, \dots, N-1$$



1. N 1D FFT along rows
2. Extract columns + FFT along temporary array
3. Transpose the result

FAST FOURIER TRANSFORM

out-of-place sequential implementation

```

 $X_{0,\dots,N-1} \leftarrow \text{ditfft2}(x, N, s):$ 
  if  $N = 1$  then
     $X_0 \leftarrow x_0$ 
  else
     $X_{0,\dots,N/2-1} \leftarrow \text{ditfft2}(x, N/2, 2s)$ 
     $X_{N/2,\dots,N-1} \leftarrow \text{ditfft2}(x+s, N/2, 2s)$ 
    for  $k = 0$  to  $(N/2)-1$  do
       $p \leftarrow X_k$ 
       $q \leftarrow \exp(-2\pi i/N k) X_{k+N/2}$ 
       $X_k \leftarrow p + q$ 
       $X_{k+N/2} \leftarrow p - q$ 
    end for
  end if

```

DFT of $(x_0, x_s, x_{2s}, \dots, x_{(N-1)s})$:

trivial size-1 DFT base case

DFT of $(x_0, x_{2s}, x_{4s}, \dots, x_{(N-2)s})$

DFT of $(x_s, x_{s+2s}, x_{s+4s}, \dots, x_{(N-1)s})$

combine DFTs of two halves:

FAST FOURIER TRANSFORM

improve performance - **in-place** sequential implementation

algorithm iterative-fft **is**

input: Array a of n complex values where n is a power of 2.

output: Array A the DFT of a .

bit-reverse-copy(a , A)

$n \leftarrow a.length$

for $s = 1$ **to** $\log(n)$ **do**

$m \leftarrow 2^s$

$\omega_m \leftarrow \exp(-2\pi i/m)$

for $k = 0$ **to** $n-1$ **by** m **do**

$\omega \leftarrow 1$

for $j = 0$ **to** $m/2 - 1$ **do**

$t \leftarrow \omega A[k + j + m/2]$

$u \leftarrow A[k + j]$

$A[k + j] \leftarrow u + t$

$A[k + j + m/2] \leftarrow u - t$

$\omega \leftarrow \omega \omega_m$

return A

FAST FOURIER TRANSFORM

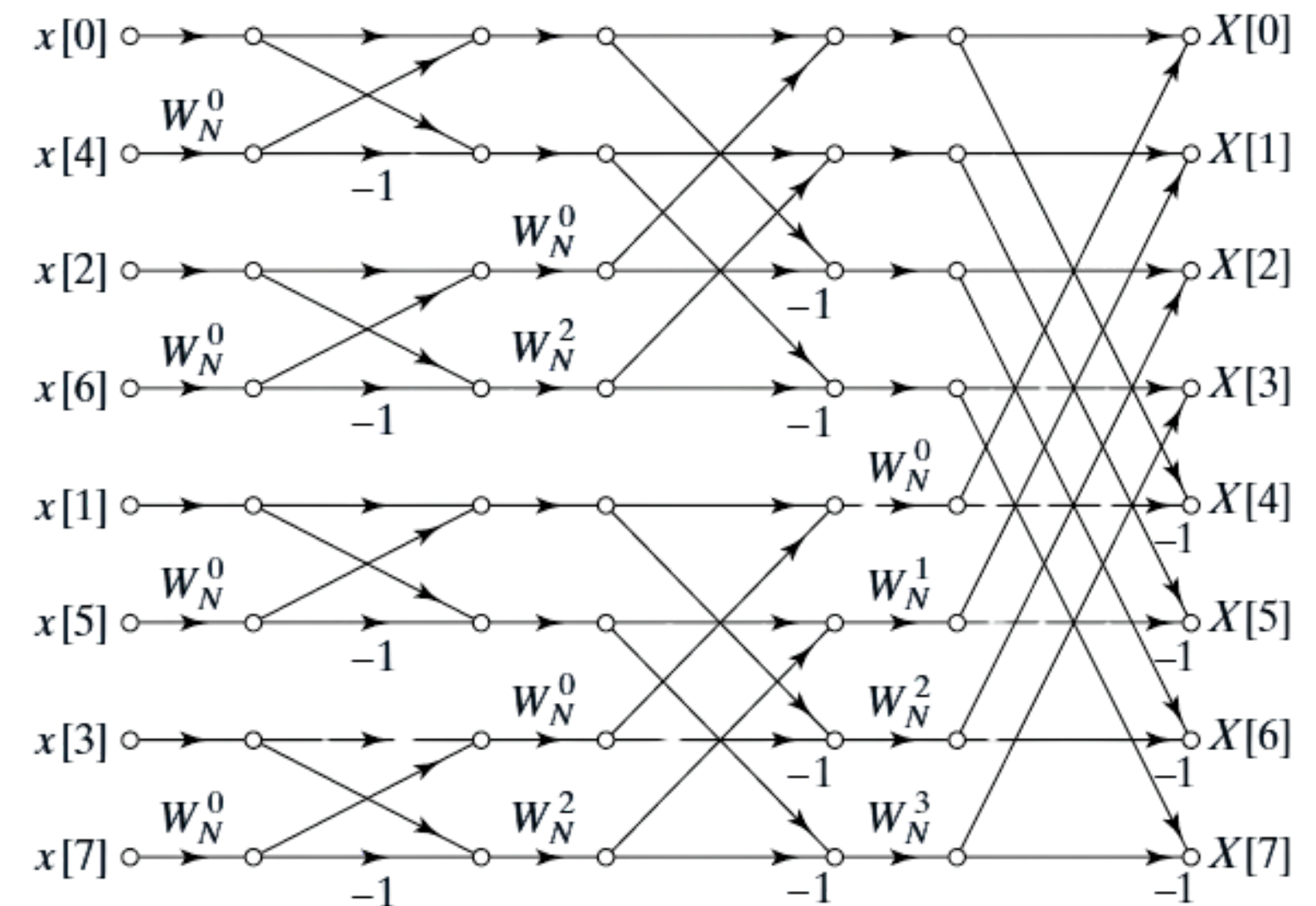
improve performance - **in-place** sequential implementation

Bit-reversal permutation

Way to organize the data so computations can be performed efficiently in place

Precompute twiddle factors

Reduces the cost of generating the coefficients



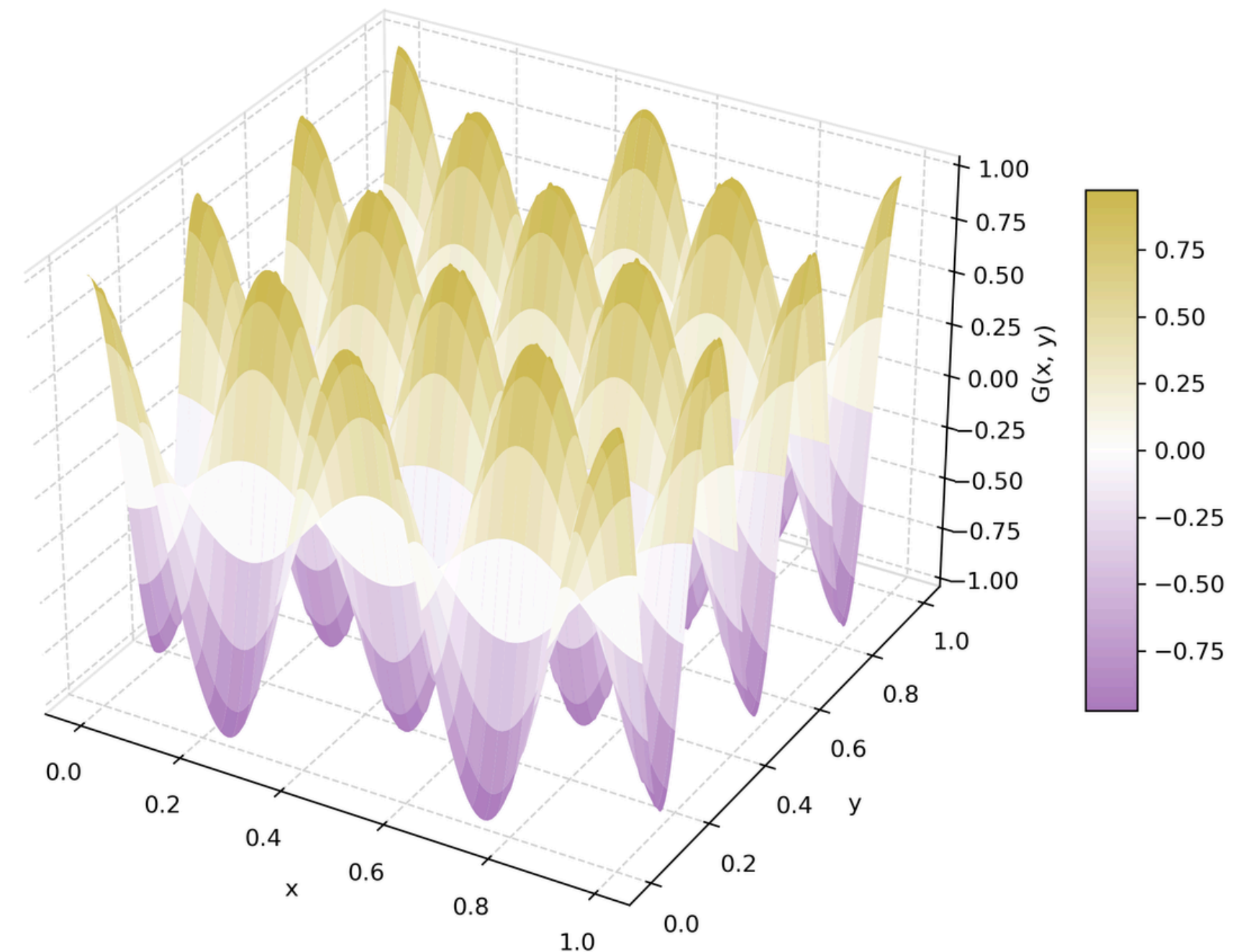
FAST FOURIER TRANSFORM

Dataset

$$G(x, y) = \cos(2\pi f_x x) \cos(2\pi f_y y)$$

$$f_x = 2$$

$$f_y = 3$$



FAST FOURIER TRANSFORM

Dataset

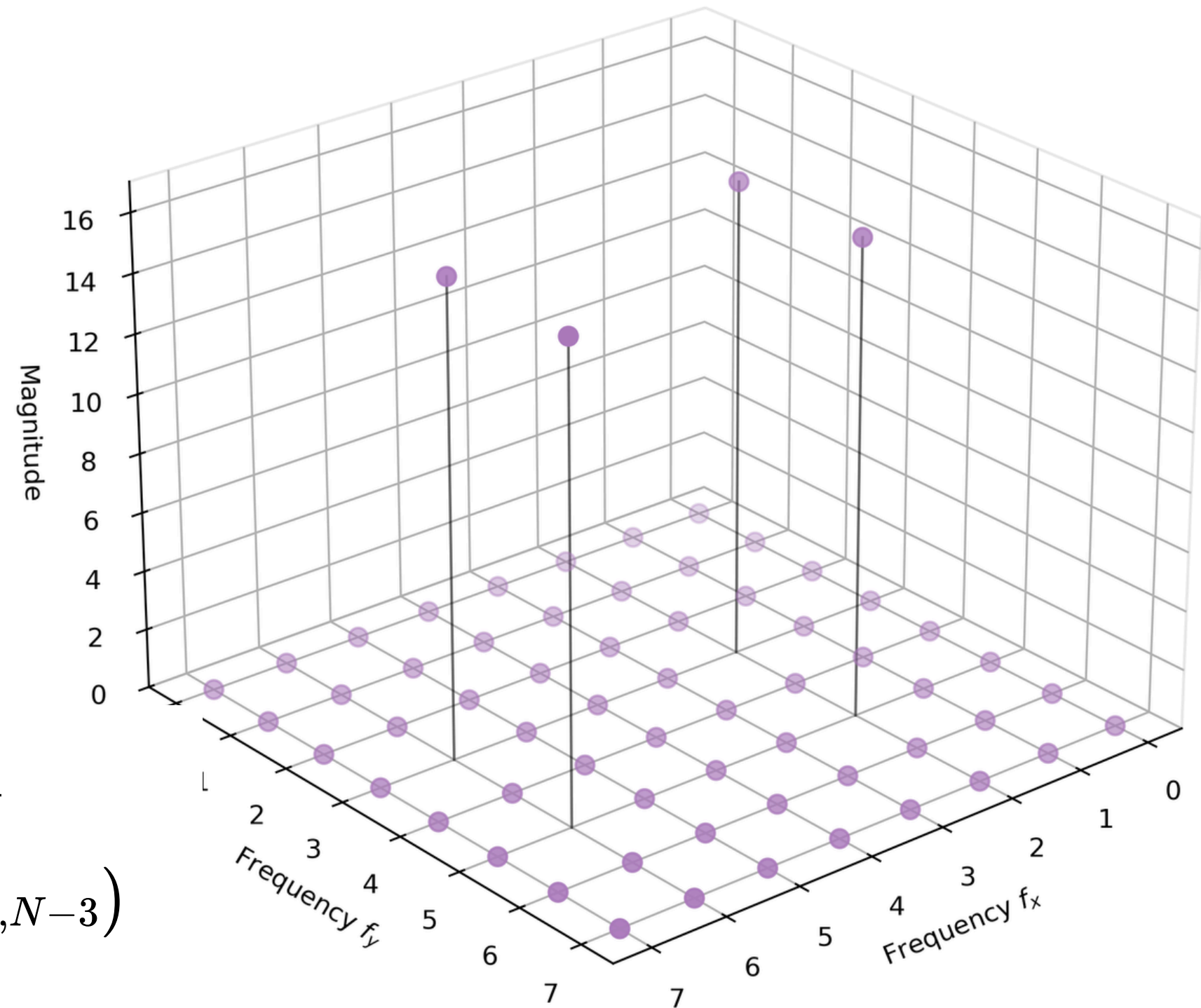
$$G(x, y) = \cos(2\pi f_x x) \cos(2\pi f_y y)$$

$$f_x = 2$$

$$f_y = 3$$

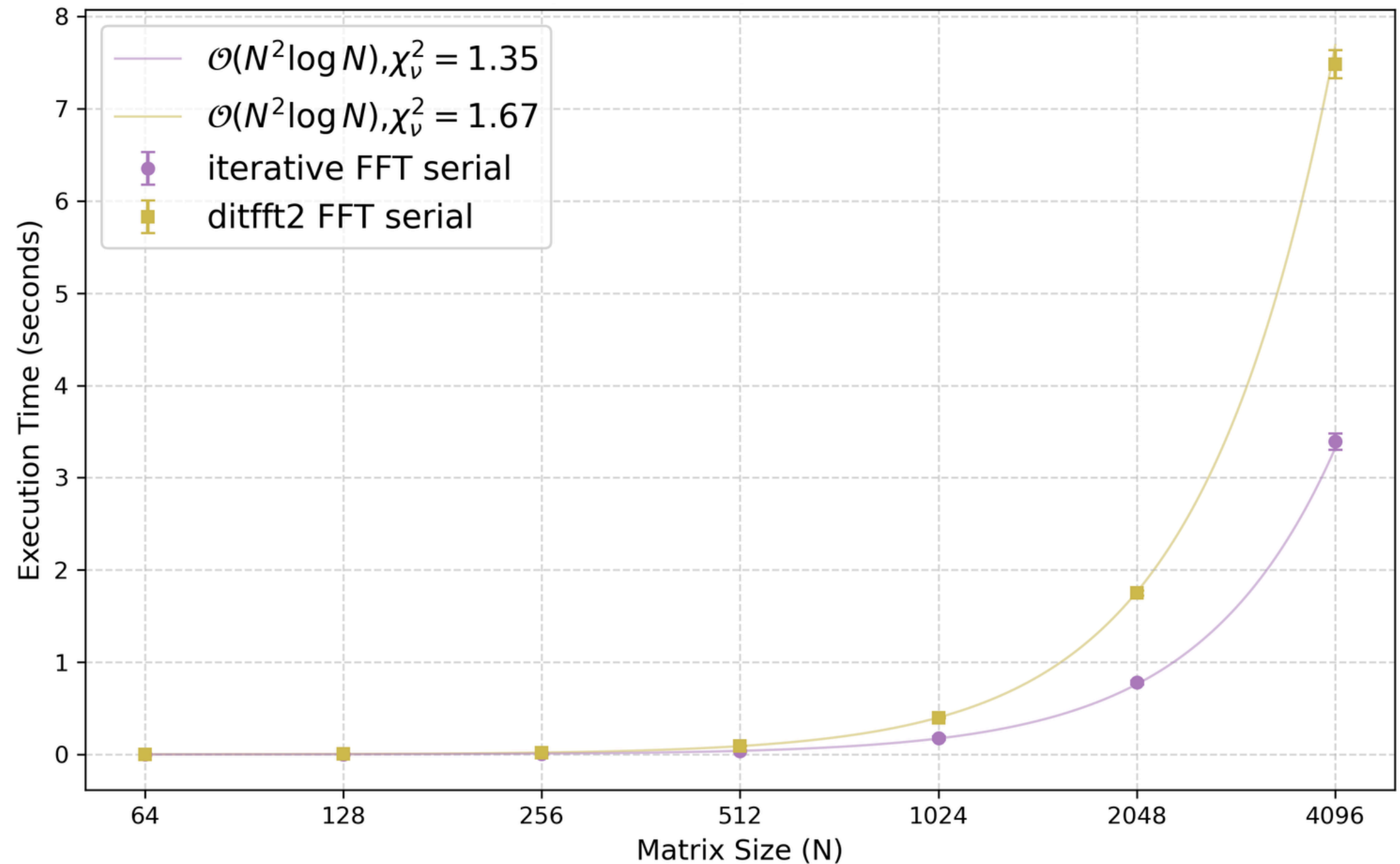
Expected output

$$G[k_x, k_y] = \frac{N^2}{4} \left(\delta_{k_x, 2} \delta_{k_y, 3} + \delta_{k_x, 2} \delta_{k_y, N-3} + \right. \\ \left. + \delta_{k_x, N-2} \delta_{k_y, 3} + \delta_{k_x, N-2} \delta_{k_y, N-3} \right)$$



FAST FOURIER TRANSFORM

Out-of-Place
vs
In-Place
Performance

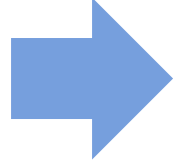


OPENMP IMPLEMENTATION

Hardware specifications (CPU)

CPU model	Intel Xeon Silver 4216 @ 2.10 GHz
Architecture	x86_64 (64 bit processor)
Logical CPUs	28
Physical cores	28
Threads per core	1

OPENMP IMPLEMENTATION

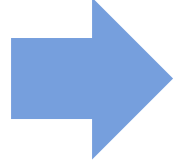
Strategy  Add compiler directives to in-place serial implementation

#pragma omp parallel for schedule(static)

Divides the iteration space evenly **before execution**, minimising scheduling overhead because every iteration performs roughly the **same amount of work**

```
#pragma omp parallel for schedule(static)
for (size_t k = 0; k < n; k++) {
    size_t r = 0;
    size_t tmp = k;
    for (size_t b = 0; b < n_bits; b++) {
        r = (r << 1) | (tmp & 1);
        tmp >>= 1;
    }
    A[r] = a[k];
}
```

OPENMP IMPLEMENTATION

Strategy  Add compiler directives to in-place serial implementation

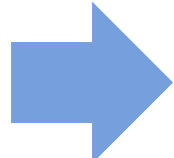
`#pragma omp critical`

Serialises this section while leaving the rest of the function parallel

Only one thread is used to allocate

```
#pragma omp critical
{
    if (twiddle_N != n) {
        free(twiddles);
        twiddles = (double complex *)malloc((n / 2)
                                           * sizeof(double complex));
        twiddle_N = n;
    }
}
```


OPENMP IMPLEMENTATION

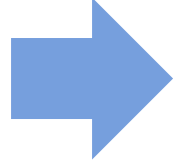
Strategy  Add compiler directives to in-place serial implementation

```
#pragma omp parallel  
for schedule(static)  
collapse(2)  
reduction(+:n_errors)
```

```
#pragma omp parallel for schedule(static) collapse(2) reduction(+:n_errors)  
for(int i=0; i<(int)N; i++){  
    for(int j=0; j<(int)N; j++){  
        double diff = 0.0;  
        if((i == fx && j == fy) || (i == fx && j == N-fy) ||  
           (i == N-fx && j == fy) || (i == N-fx && j == N-fy)){  
            diff = fabs(X[i * N + j] - pow(N, 2)/4);  
        } else {  
            diff = X[i * N + j];  
        }  
    }  
}
```

collapse(2): flattens the nested loops into one iteration space and distribute the iterations across threads

OPENMP IMPLEMENTATION

Strategy  Add compiler directives to in-place serial implementation

```
#pragma omp parallel  
for schedule(static)  
collapse(2)  
reduction(+:n_errors)
```

```
#pragma omp parallel for schedule(static) collapse(2) reduction(+:n_errors)  
for(int i=0; i<(int)N; i++){  
    for(int j=0; j<(int)N; j++){  
        double diff = 0.0;  
        if((i == fx && j == fy) || (i == fx && j == N-fy) ||  
           (i == N-fx && j == fy) || (i == N-fx && j == N-fy)){  
            diff = fabs(X[i * N + j] - pow(N, 2)/4);  
        } else {  
            diff = X[i * N + j];  
        }  
    }  
}
```

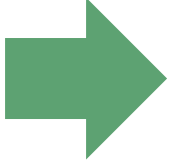
reduction(+:n_errors): gives every thread a **private copy** of n_errors . After the loop finishes, OpenMP sums the private values into the shared variable

CUDA IMPLEMENTATION

Hardware specifications (GPU)

GPU model	NVIDIA Tesla T4
Architecture	Turing
GPU Memory	15 GB GDDR6
Shared Memory	64 kB
Streaming Multiprocessors	40
CUDA Cores	2560
Maximum threads per block	1024

CUDA IMPLEMENTATION

Functions  CUDA Kernels

Grid : $\text{dim3}(N/TPB, N)$
Block : $\text{dim3}(TPB)$

Each thread handles one element tid

Bit-reversal only permutes elements within a row, so rows never need to interact

```
__global__ void kernel_bit_reverse(cuDoubleComplex *A, unsigned int n,
                                   unsigned int n_bits){
    unsigned int tid = blockIdx.x * blockDim.x + threadIdx.x;
    unsigned int row = blockIdx.y;

    // compute bit-reversal of tid
    unsigned int r = 0;
    unsigned int tmp = tid;
    for (int b = 0; b < (int)n_bits; b++) {
        r = (r << 1) | (tmp & 1);
        tmp >>= 1;
    }

    if (r > tid) { // to avoid double swapping
        cuDoubleComplex t = A[row * n + tid];
        A[row * n + tid] = A[row * n + r];
        A[row * n + r] = t;
    }
}
```

CUDA IMPLEMENTATION

Grid : $\text{dim3}(N/TPB, N)$
Block : $\text{dim3}(TPB)$

TPB = 256

Each thread handles one element tid

Bit-reversal only permutes elements within a row, so rows never need to interact

```
__global__ void kernel_bit_reverse(cuDoubleComplex *A, unsigned int n,
                                   unsigned int n_bits){
    unsigned int tid = blockIdx.x * blockDim.x + threadIdx.x;
    unsigned int row = blockIdx.y;

    // compute bit-reversal of tid
    unsigned int r = 0;
    unsigned int tmp = tid;
    for (int b = 0; b < (int)n_bits; b++) {
        r = (r << 1) | (tmp & 1);
        tmp >>= 1;
    }

    if (r > tid) { // to avoid double swapping
        cuDoubleComplex t = A[row * n + tid];
        A[row * n + tid] = A[row * n + r];
        A[row * n + r] = t;
    }
}
```


CUDA IMPLEMENTATION

Grid : $\text{dim3}((N/2)/\text{TPB}, N)$
 Block : $\text{dim3}(\text{TPB})$

Each thread handles one pair of elements

$N/2$ threads are enough to cover an entire row

$\log_2(N)$ launches to complete the butterfly step

```
__global__ void kernel_butterfly(cuDoubleComplex *A, unsigned int n,
                                unsigned int m, unsigned int step){
    unsigned int tid = blockIdx.x * blockDim.x + threadIdx.x;
    unsigned int row = blockIdx.y;
    unsigned int half = m >> 1;

    unsigned int k = (tid / half) * m; // start of the group
    unsigned int j = tid % half;       // which butterfly within the group

    // twiddle factor  $W_n^{\{j * \text{step}\}} = e^{\{-2\pi i * j * \text{step} / n\}}$ 
    double angle = -2.0 * M_PI * (double)(j * step) / (double)n;
    double c, s;
    sincos(angle, &s, &c);
    cuDoubleComplex w = make_cuDoubleComplex(c, s);

    cuDoubleComplex u = A[row * n + k + j];
    cuDoubleComplex t = cuCmul(w, A[row * n + k + j + half]);

    A[row * n + k + j] = cuCadd(u, t);
    A[row * n + k + j + half] = cuCsub(u, make_cuDoubleComplex(cuCreal(t), cuCimag(t)));
}
```


CUDA IMPLEMENTATION

Grid : $\text{dim3}(N/TILE, N/TILE)$

Block : $\text{dim3}(TILE, TILE)$

Naive global-to-global memory transpose

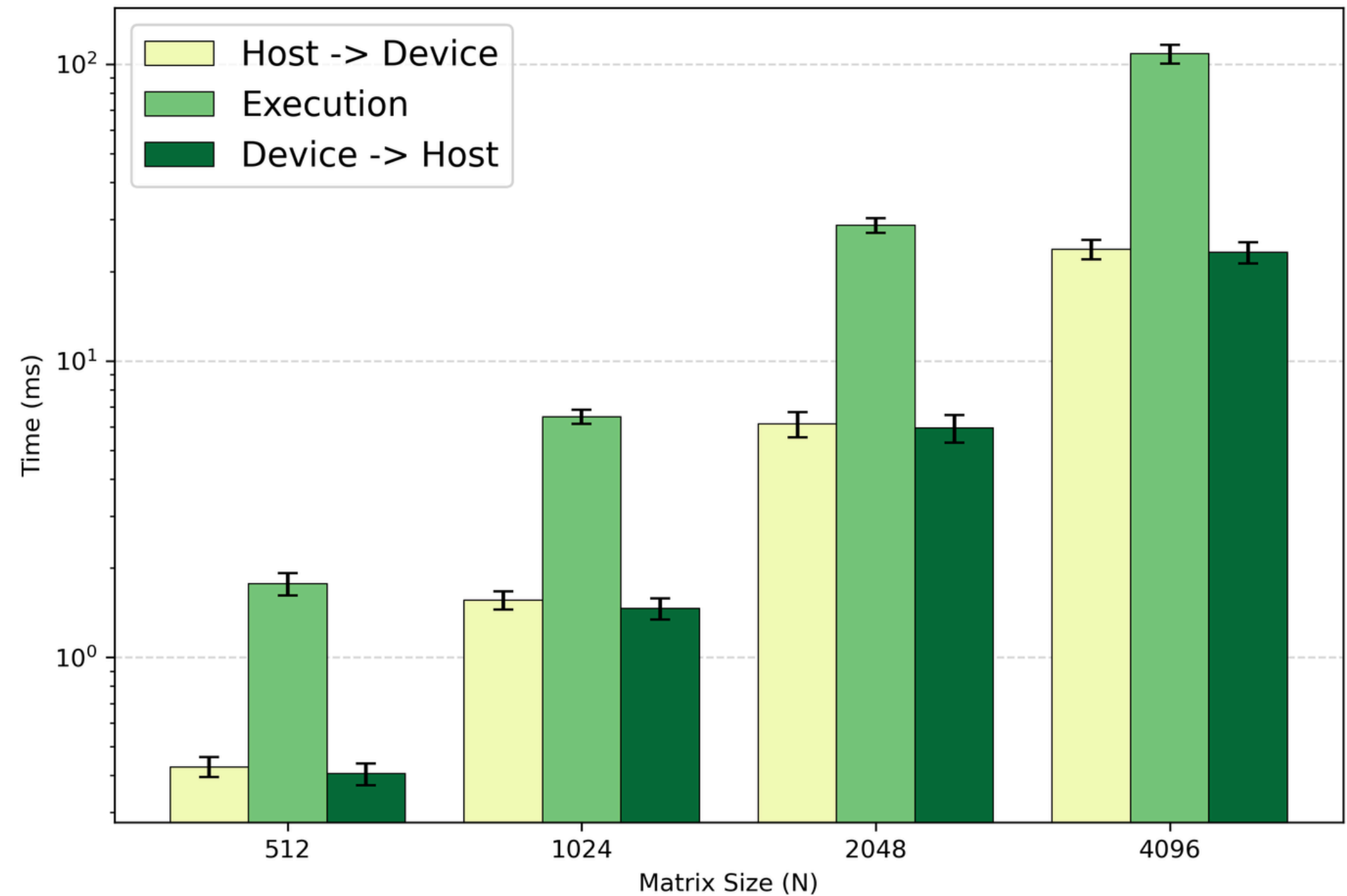
$TILE = 32$ (exact warp size) to maximize the work per block launch

A TILE larger than 32 would need > 1024 threads per block ($TILE * TILE$), which exceeds the hardware limit

```
__global__ void kernel_transpose(const cuDoubleComplex *A,  
                                cuDoubleComplex *A_T,  
                                unsigned int n)  
{  
    unsigned int col = blockIdx.x * blockDim.x + threadIdx.x;  
    unsigned int row = blockIdx.y * blockDim.y + threadIdx.y;  
    if (row < n && col < n)  
        A_T[col * n + row] = A[row * n + col];  
}
```

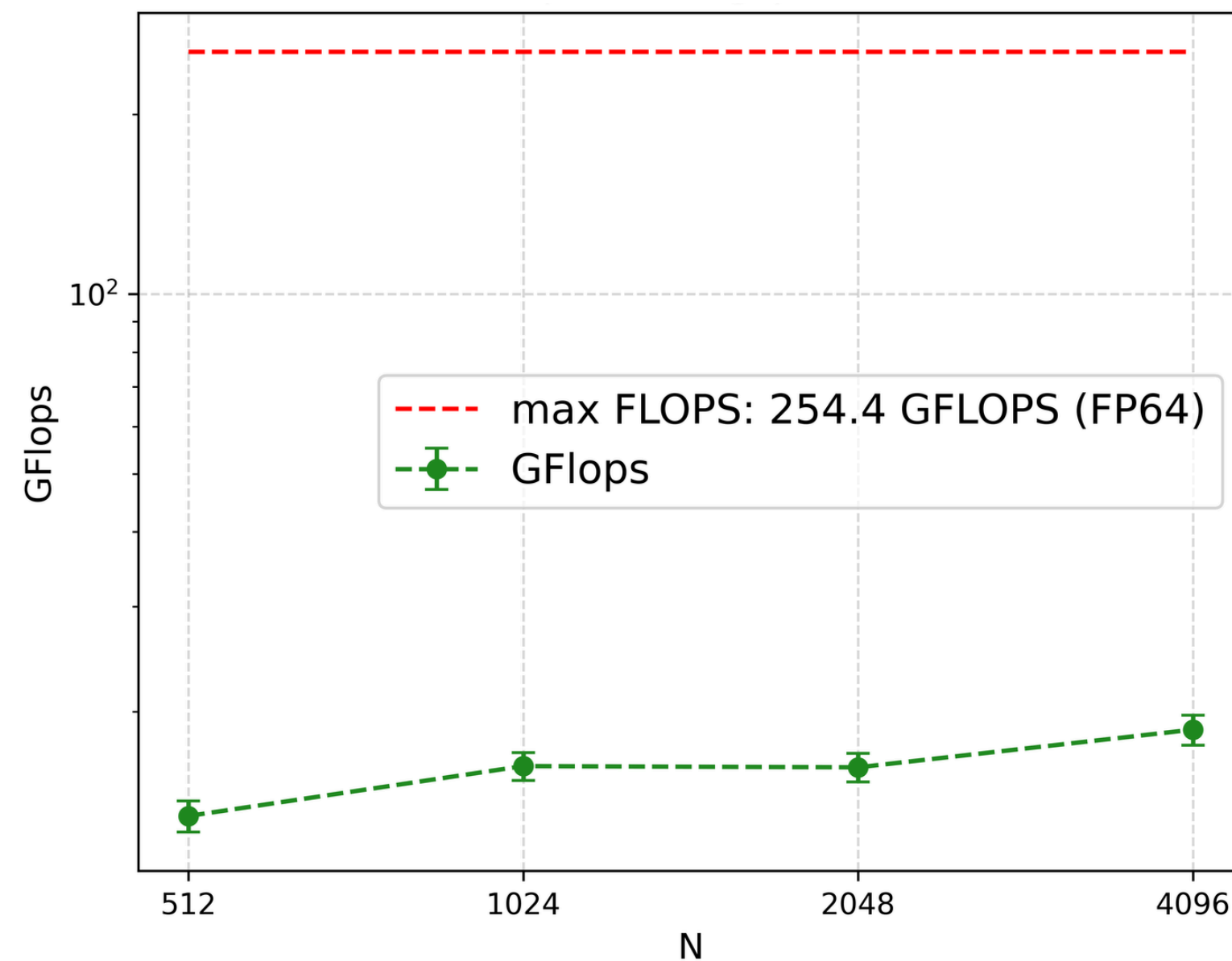
CUDA IMPLEMENTATION

CUDA Timing inspection

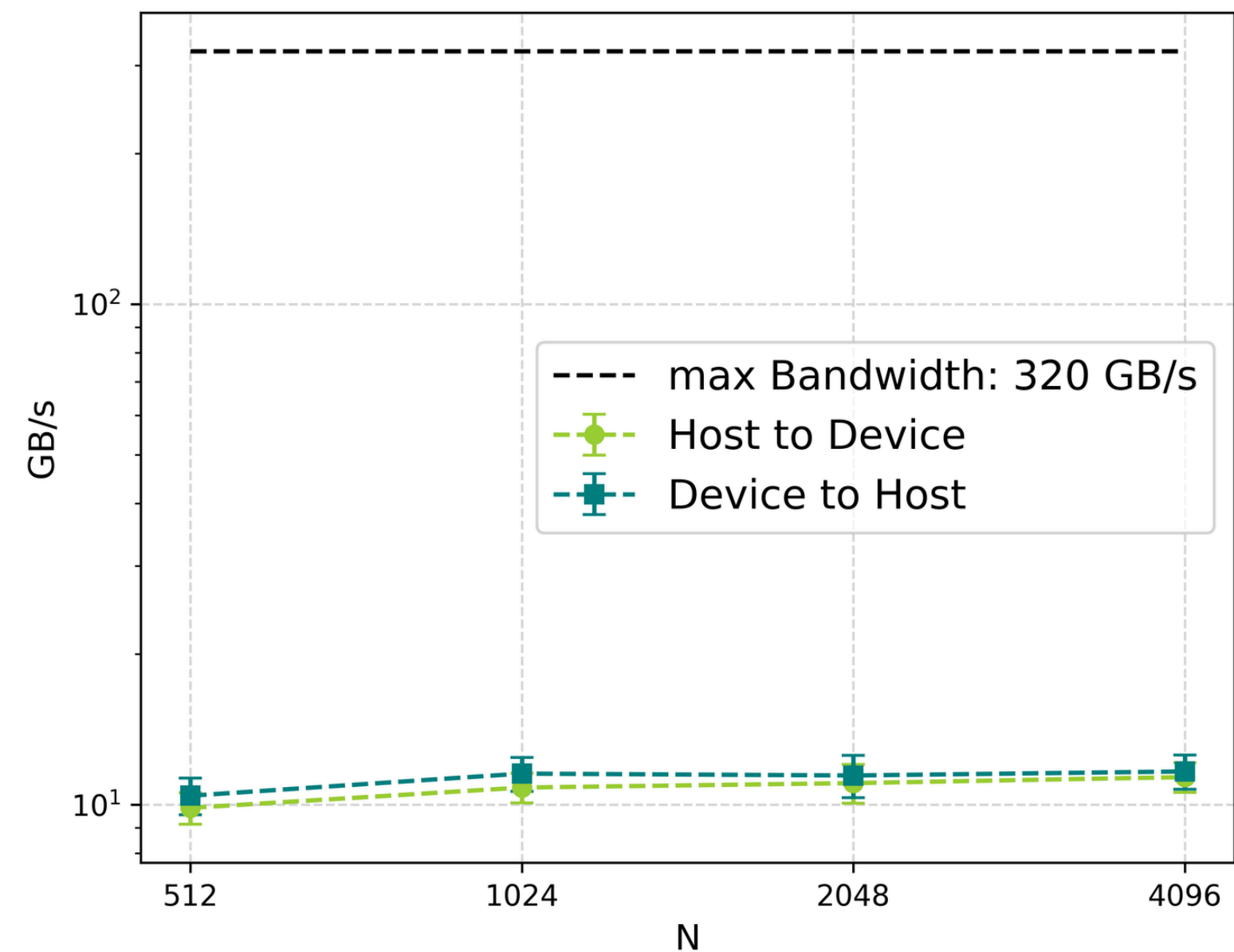


CUDA IMPLEMENTATION

GFLOPS vs N

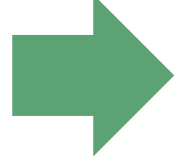


Bandwidth vs N



CUDA IMPLEMENTATION

Shared Memory

kernel_bit_reverse + *kernel_butterfly*  fused in one

The whole row lives
in shared memory
for the entire FFT

Drawback: exceeds
shared memory
limit for $N \geq 4096$
in Double precision
computation

```
extern __shared__ cuDoubleComplex smem[];

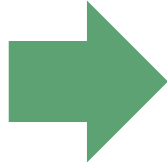
__global__ void kernel_fft_row_shared(cuDoubleComplex *A, unsigned int n,
                                     unsigned int n_bits, unsigned int rows_per_block){
    unsigned int half_n = n >> 1;
    unsigned int local_row = threadIdx.y;
    unsigned int row = blockIdx.x * rows_per_block + local_row;
    unsigned int tid = threadIdx.x;

    cuDoubleComplex *srow = smem + local_row * n;

    // load: global -> shared
    srow[tid] = A[row * n + tid];
    srow[tid + half_n] = A[row * n + tid + half_n];
    __syncthreads();
}
```

CUDA IMPLEMENTATION

Shared Memory

kernel_transpose  tiled in shared memory

Classic efficient matrix transpose pattern in shared memory from NVIDIA

```
__global__ void kernel_transpose_shared(const cuDoubleComplex *A,
                                       cuDoubleComplex *A_T, unsigned int n){
    __shared__ cuDoubleComplex tile[TILE_DIM][TILE_DIM + 1];

    unsigned int x = blockIdx.x * TILE_DIM + threadIdx.x;
    unsigned int y = blockIdx.y * TILE_DIM + threadIdx.y;

    for (unsigned int j = 0; j < TILE_DIM; j += BLOCK_ROWS) {
        if (x < n && (y + j) < n)
            tile[threadIdx.y + j][threadIdx.x] = A[(y + j) * n + x];
    }

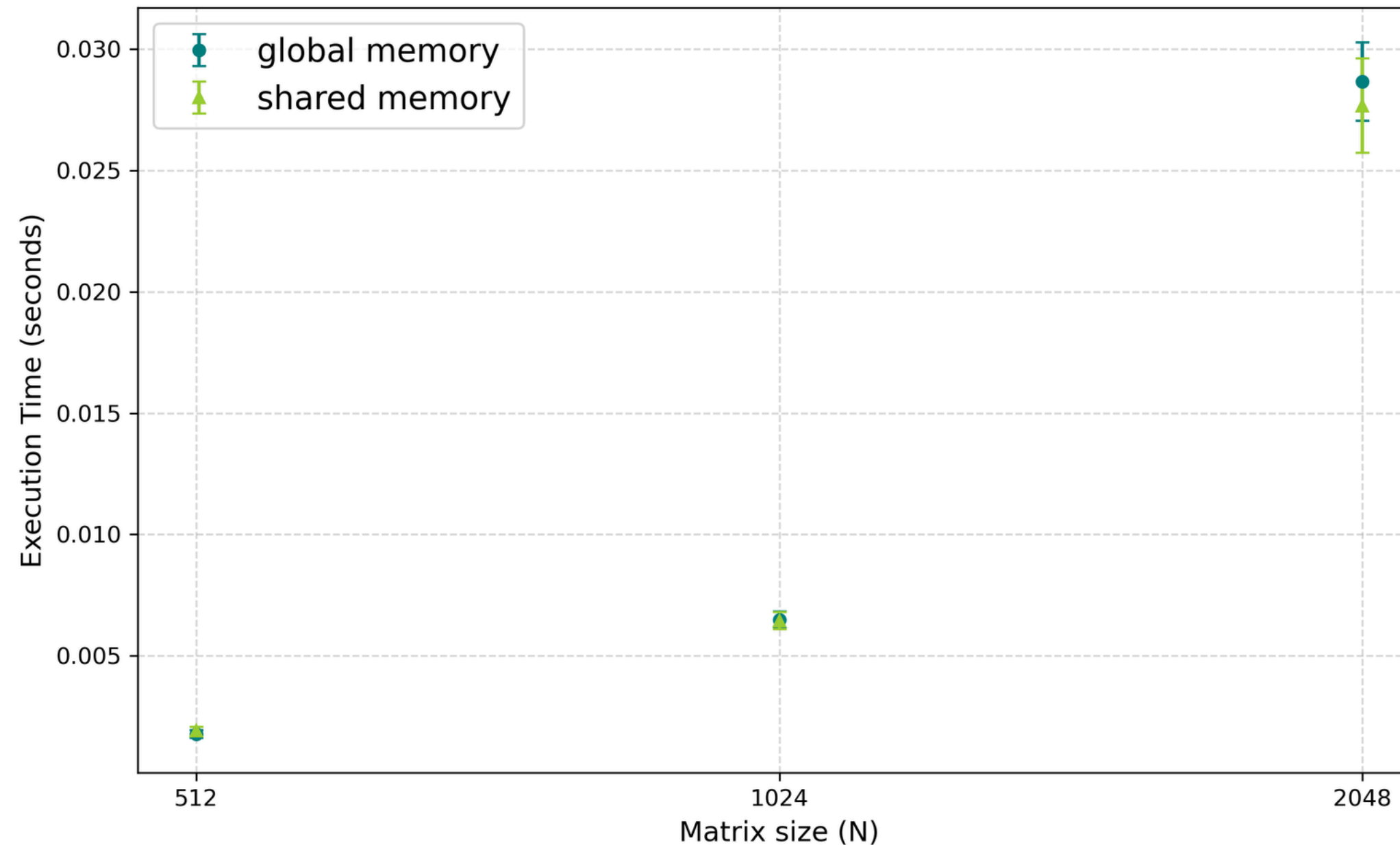
    __syncthreads();

    x = blockIdx.y * TILE_DIM + threadIdx.x;    // transpose block offset
    y = blockIdx.x * TILE_DIM + threadIdx.y;

    for (unsigned int j = 0; j < TILE_DIM; j += BLOCK_ROWS) {
        if (x < n && (y + j) < n){
            A_T[(y + j) * n + x] = tile[threadIdx.x][threadIdx.y + j];
        }
    }
}
```

CUDA IMPLEMENTATION

Global vs Shared Memory Performance

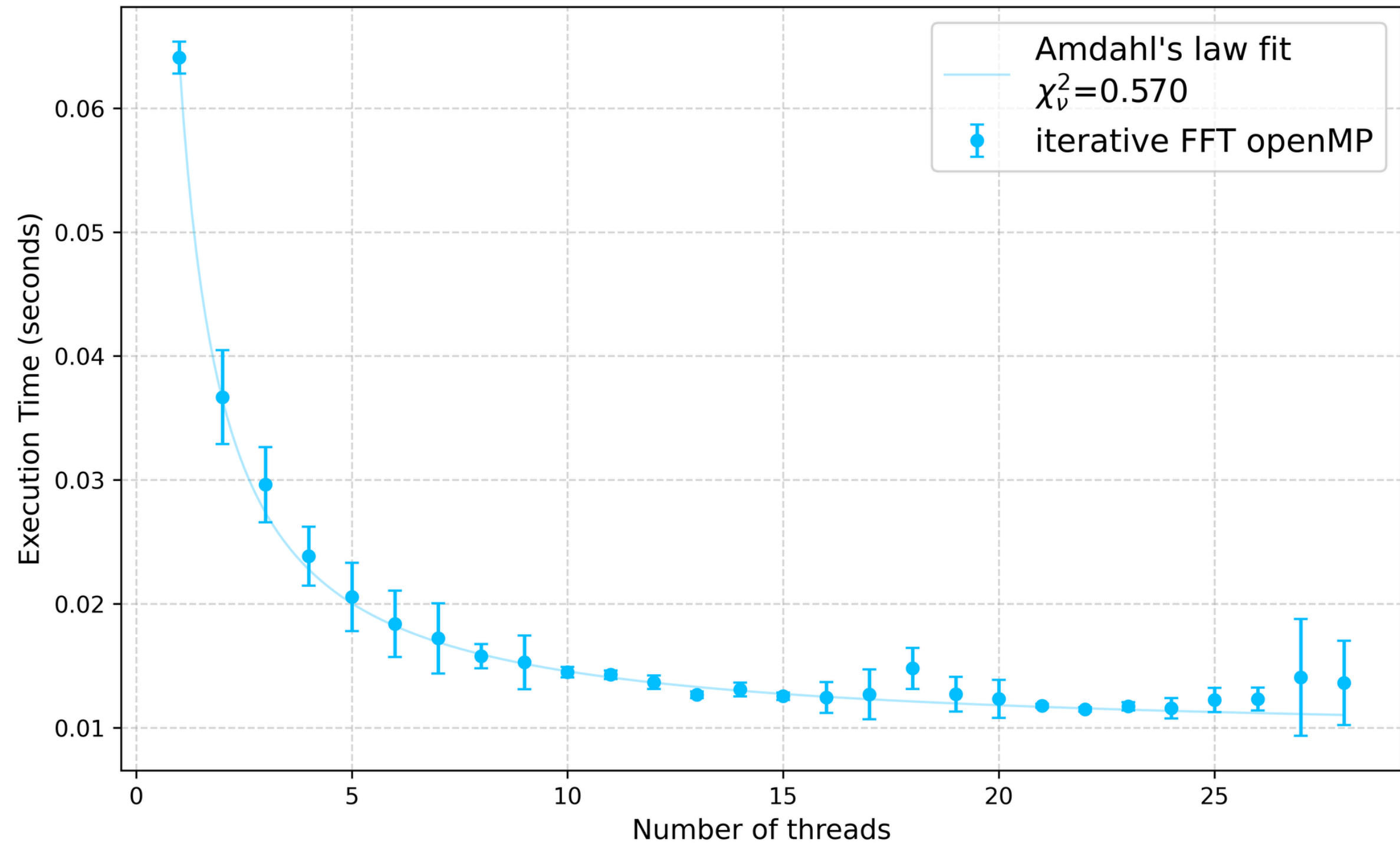


BENCHMARKS

OpenMP

Computation Time
vs Number of
Threads

$N = 512$

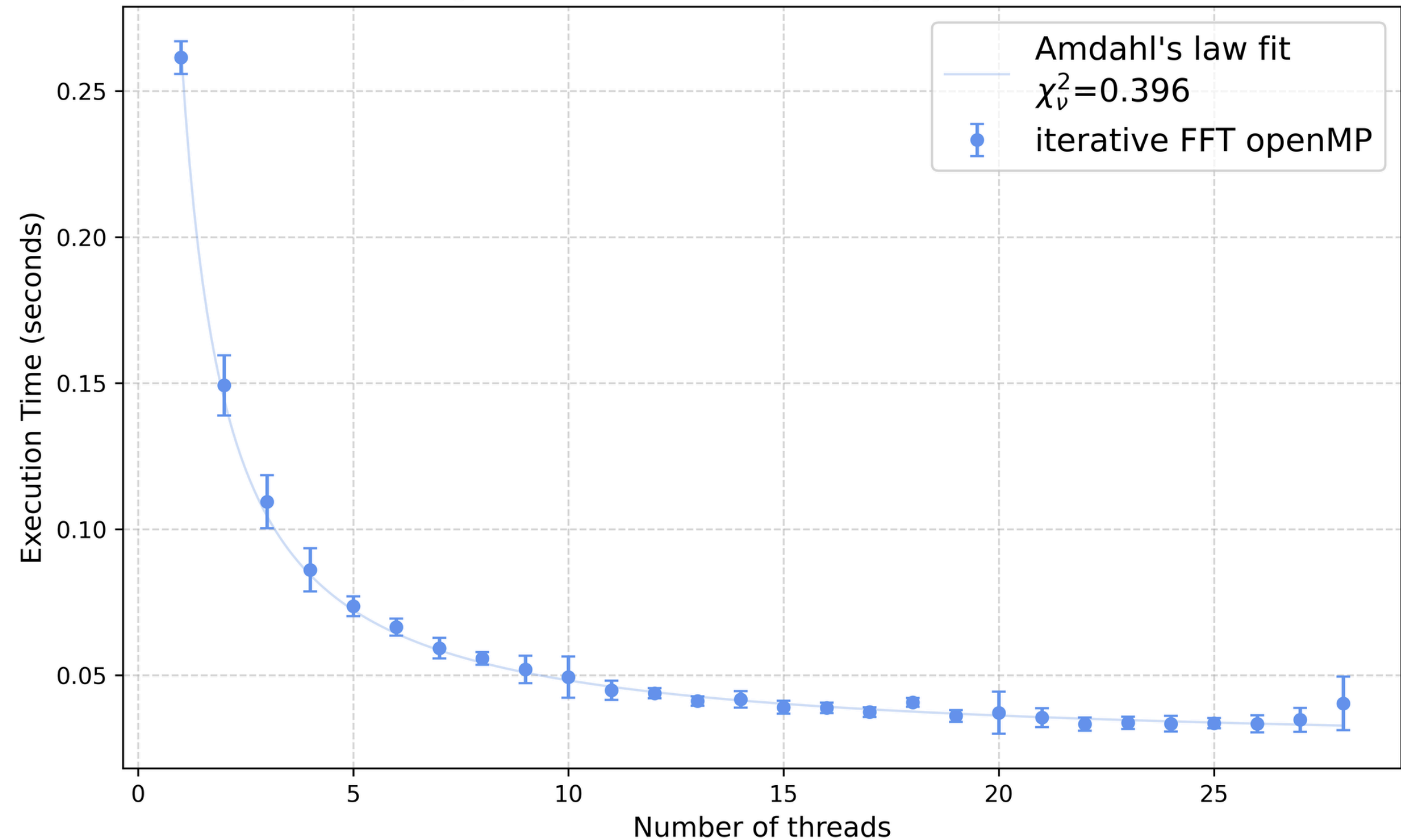


BENCHMARKS

OpenMP

Computation Time
vs Number of
Threads

$N = 1024$

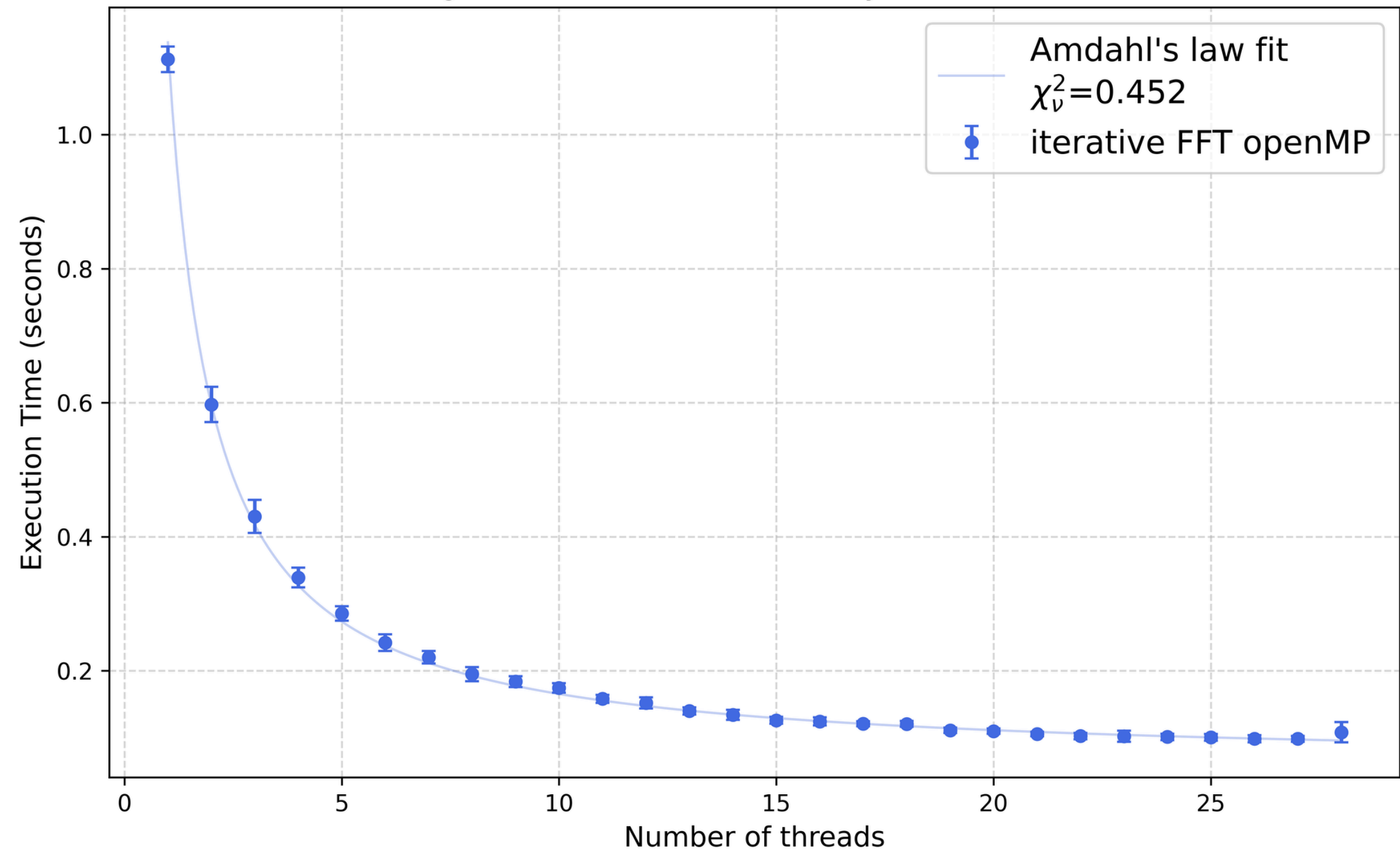


BENCHMARKS

OpenMP

Computation Time
vs Number of
Threads

$N = 2048$

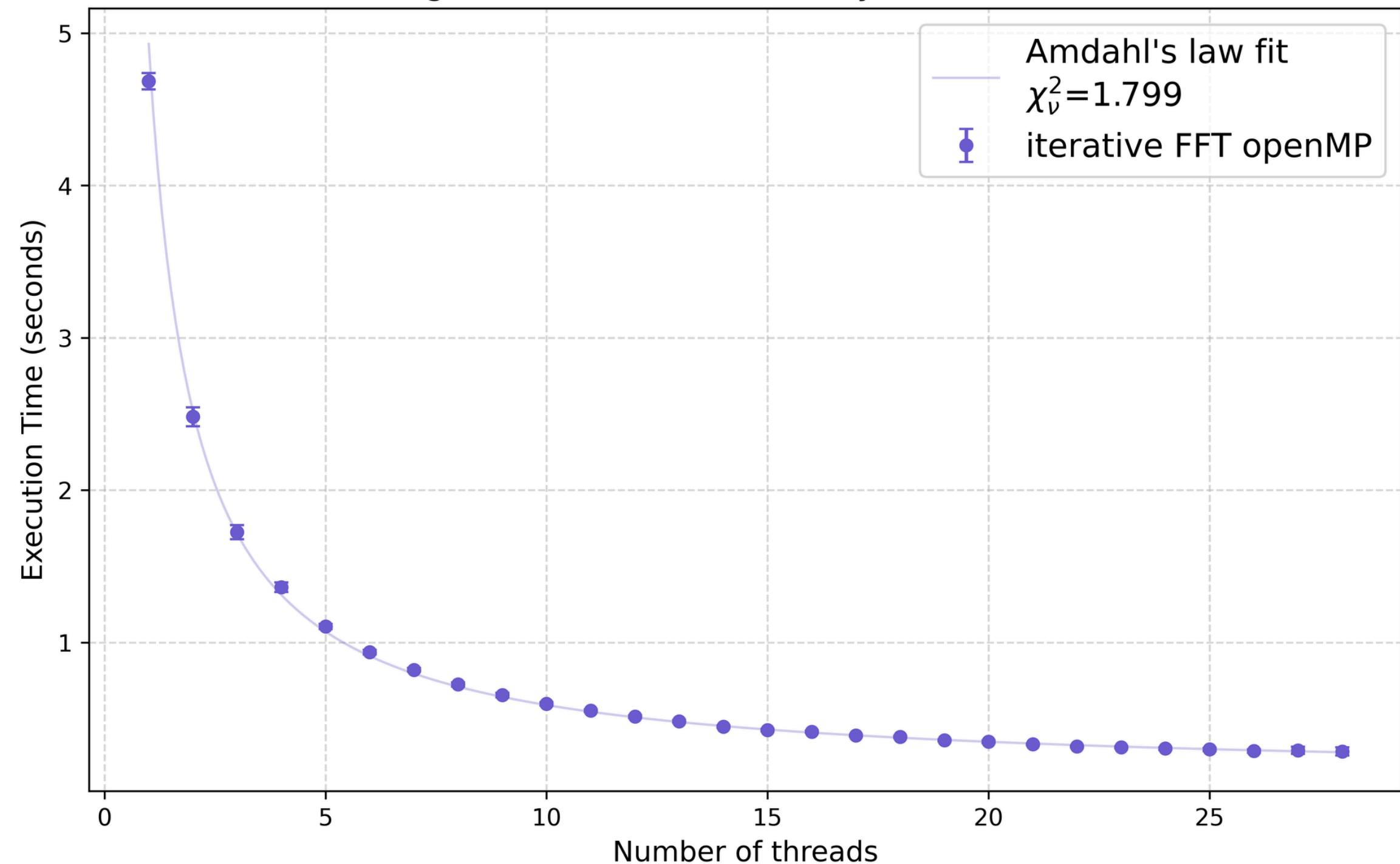


BENCHMARKS

OpenMP

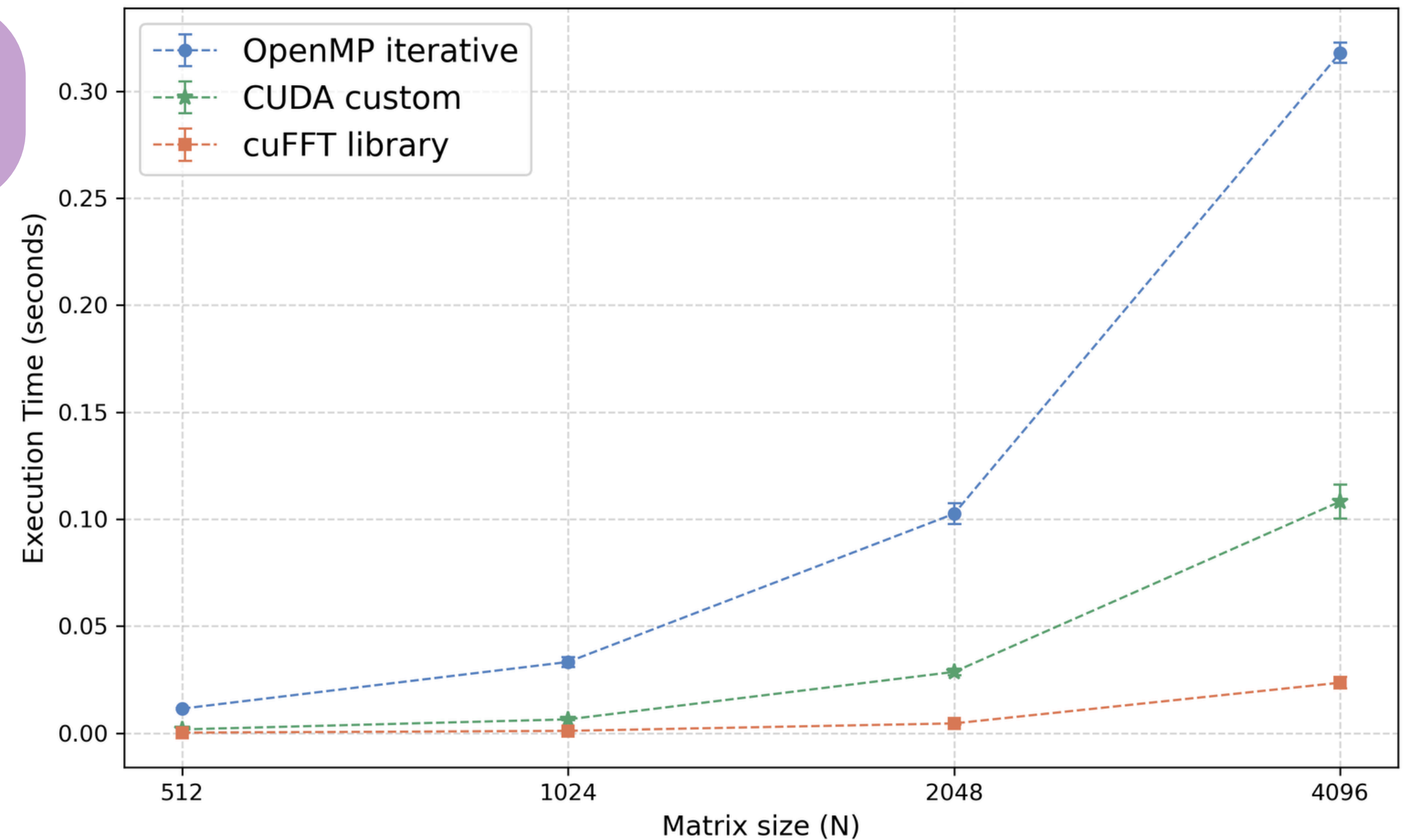
Computation Time
vs Number of
Threads

$N = 4096$



BENCHMARKS

Comparison:
OpenMP - CUDA - cuFFT



CONCLUSIONS

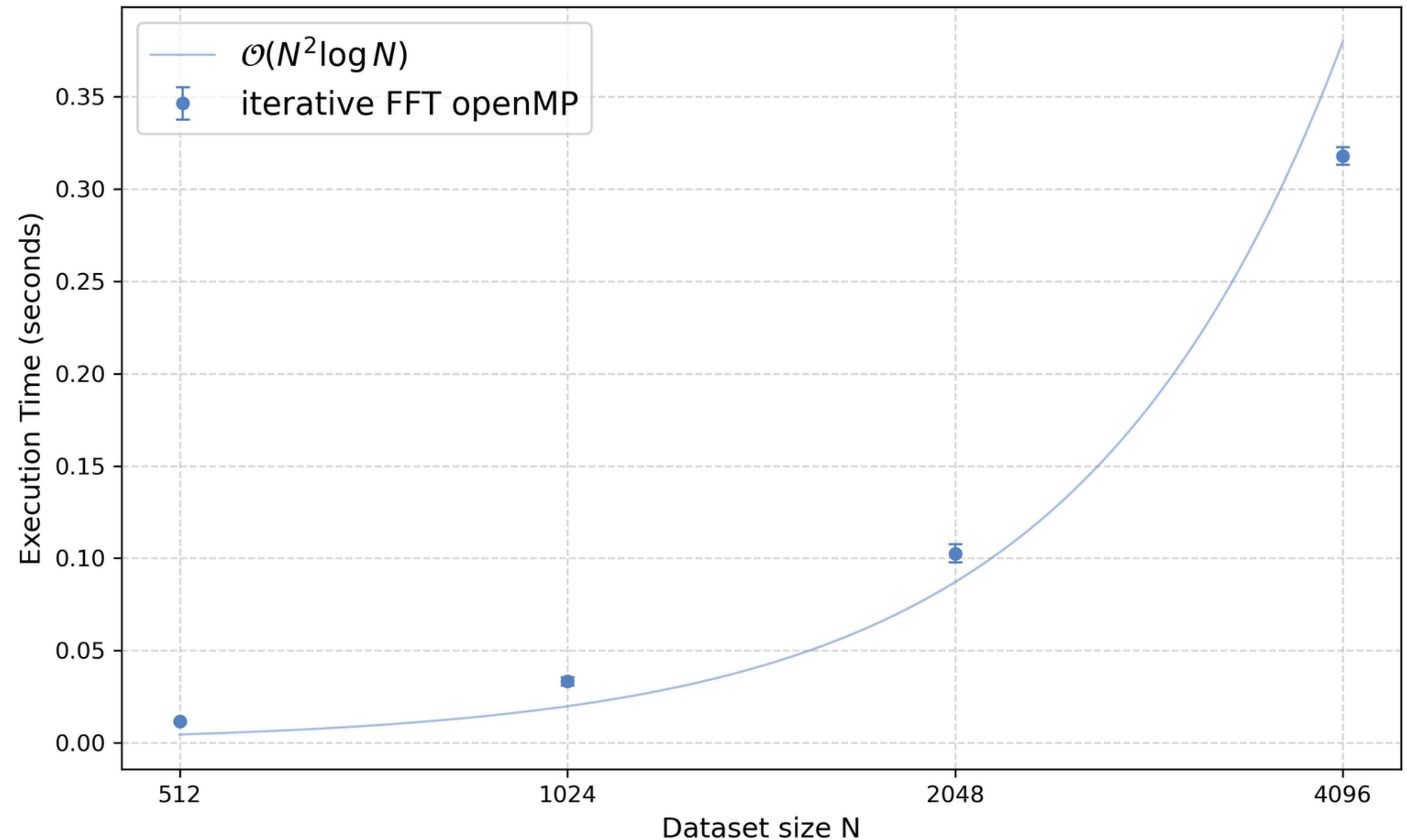
- OpenMP computation time scales with the number of threads following Amdahl's law
- CUDA custom computation does not gain speed changing from Global to Shared Memory
- $T_{\text{OpenMP}} > T_{\text{CUDA}} > T_{\text{cuFFT}}$
- Many applications (e.g. Image Processing)

BACKUP

OpenMP

Computation Time
vs Matrix size N

$NTh = 22$



BACKUP

CUDA

Computation Time
vs Matrix size N

